

Alunos: Ana Carolina, João Alvin, Tiago Segatti

Disciplina: Análise de Algoritmos

Lista 06

Questão 1 - Busca Binária

O algoritmo recebe um vetor ordenado e um valor a ser buscado. A cada chamada recursiva, calcula o índice do meio do intervalo e compara com o valor. Se forem iguais, retorna o índice. Se o valor for menor que o meio, repete o processo apenas na metade esquerda. Se for maior, na metade direita. O caso base é quando o intervalo fica vazio ($\text{início} > \text{fim}$), retornando -1 para indicar que o valor não existe. A cada chamada o problema é reduzido à metade, resultando em complexidade $O(\log n)$.

Questão 2 - Elemento igual ao índice

Segue a mesma lógica da busca binária, mas em vez de comparar o elemento com um valor externo, compara o elemento com o próprio índice onde ele está. Se $A[\text{meio}] == \text{meio}$, encontrou. Se $A[\text{meio}] > \text{meio}$, como o vetor é ordenado, todos os elementos à direita terão uma diferença ainda maior entre valor e índice, então a resposta só pode estar à esquerda. O inverso vale para $A[\text{meio}] < \text{meio}$. Complexidade $O(\log n)$.

Questão 3 - Inverter String

A cada chamada, os caracteres das extremidades do intervalo atual são trocados entre si. Depois, o algoritmo chama a si mesmo para o intervalo interno, avançando uma posição de cada lado. O caso base é quando as posições se cruzam ($\text{início} \geq \text{fim}$), o que significa que todos os caracteres já foram trocados. É $O(n)$ pois cada caractere é visitado exatamente uma vez.

Questão 4 - Elemento Majoritário

Essa é a mais elaborada. Como a restrição proíbe comparações de maior/menor, não é possível ordenar o vetor para contar. A solução divide o vetor ao meio e, recursivamente, encontra o candidato majoritário de cada metade. O caso base é um único elemento, que é sempre candidato de si mesmo. Com os dois candidatos em mãos, conta-se quantas vezes cada um aparece no intervalo inteiro usando apenas igualdade, também de forma recursiva. Se algum aparecer mais de $n/2$ vezes, ele é o majoritário. A lógica é válida porque se um elemento domina o vetor inteiro, ele obrigatoriamente domina pelo menos uma das metades. Complexidade $O(n \log n)$, pois a cada nível da recursão faz-se $O(n)$ comparações ao contar as ocorrências.